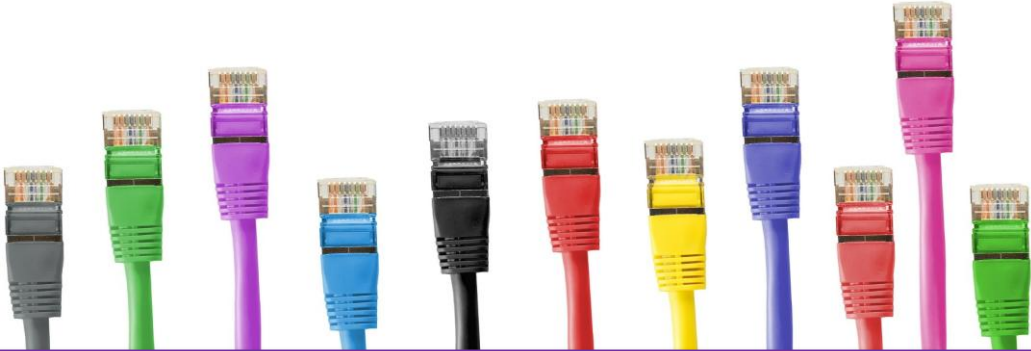




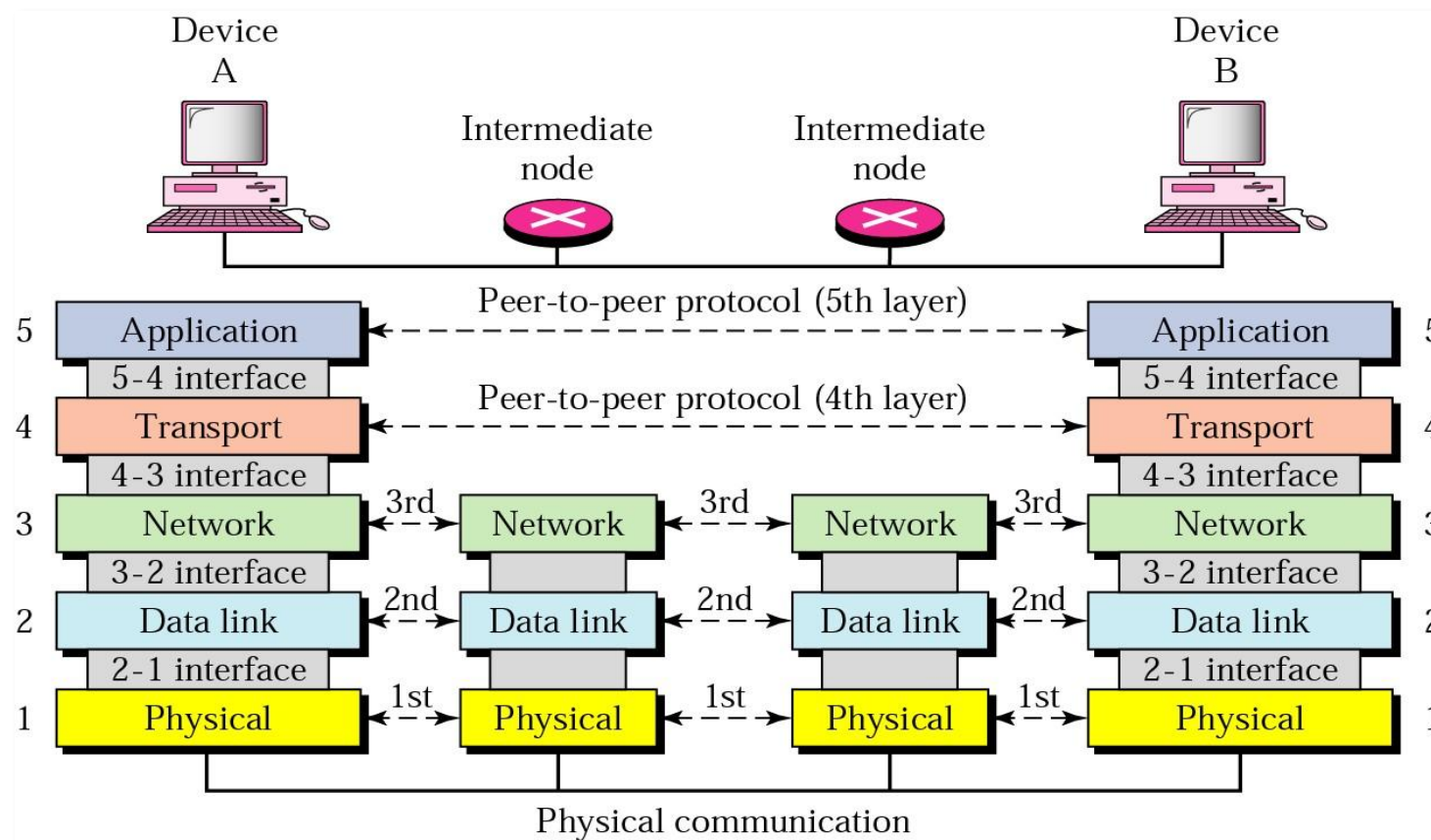
SOF203 Fundamentals of Network Technology

Lecture 5 – Transport Layer (Part 1)

Xiamen University Malaysia



Recap



Behrouz A. Forouzan, "Data Communications and Networking"

Objectives

- understand principles behind transport layer services:
 - reliable data transfer
 - flow control
 - congestion control
- learn about transport layer protocols:
 - UDP: connectionless transport
 - TCP: connection-oriented transport

Contents

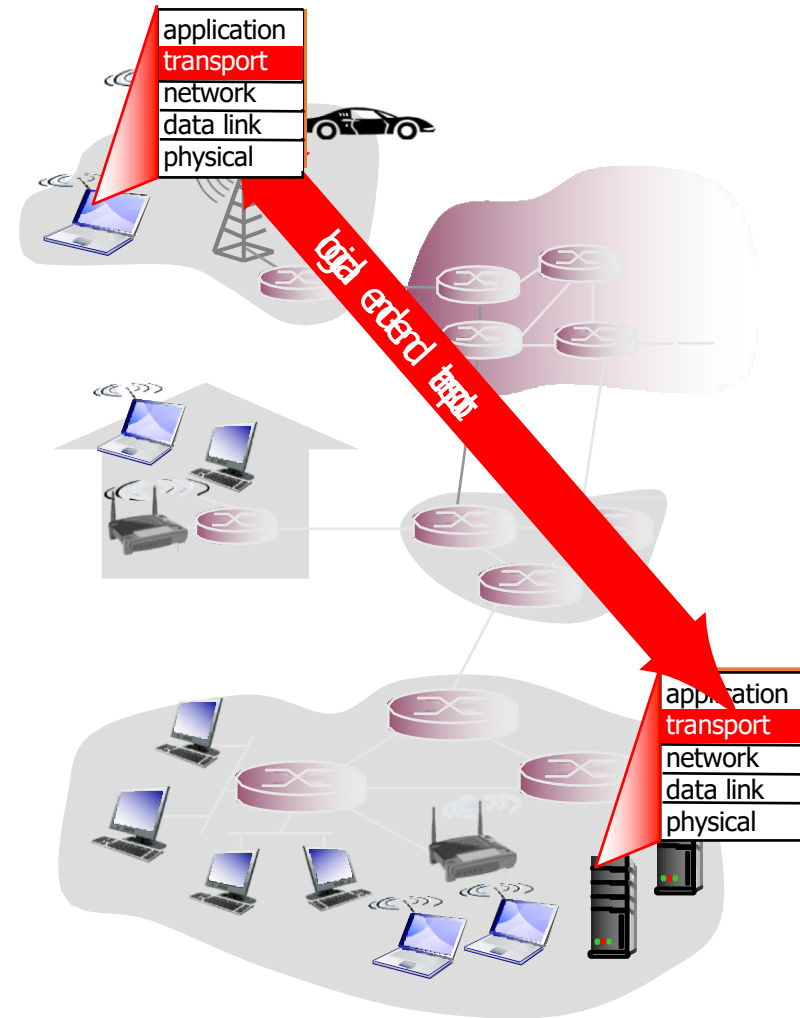
- Transport-layer services
- Principles of reliable data transfer
- Stop-and-Wait
- Go-Back-N
- Selective-Repeat

Contents

- Transport-layer services
- Principles of reliable data transfer
- Stop-and-Wait
- Go-Back-N
- Selective-Repeat

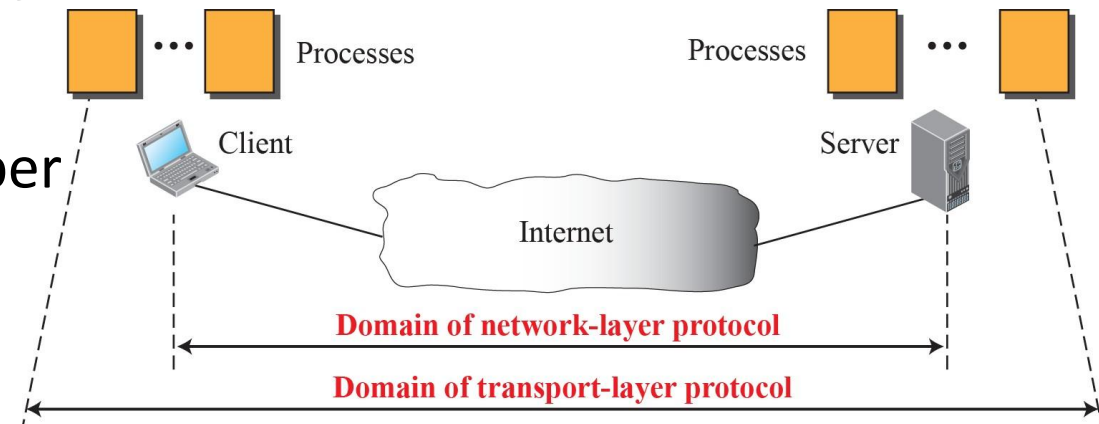
Transport Services and Protocols

- provide *logical communication* between *application processes* running on different hosts
- transport protocols run in end systems
 - sender: breaks app messages into *segments*, passes to network layer
 - receiver: reassembles segments into messages, passes to app layer



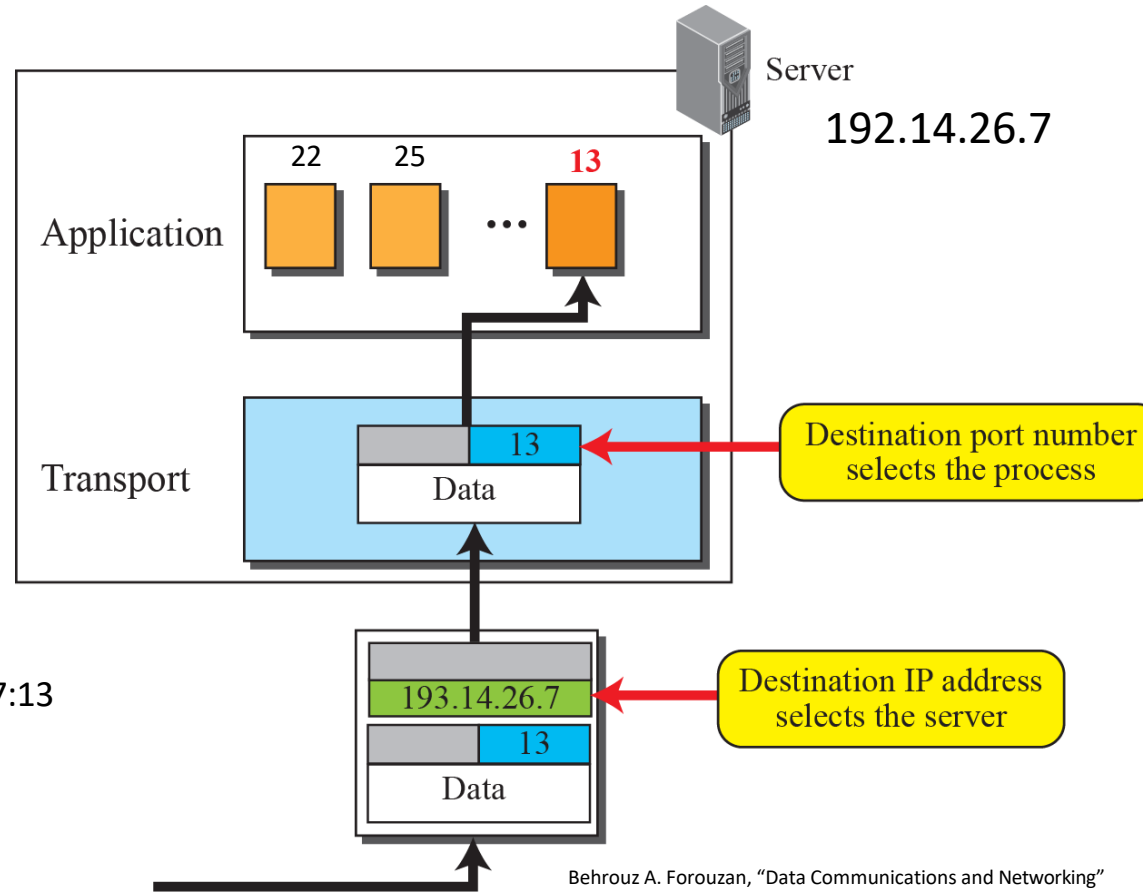
Transport vs. Network Layer

- **network layer**: logical communication between **hosts**
 - Use 32-bit **IP address** to identify host
- **transport layer**: logical communication between **processes**
 - Use 16-bit **port number** to identify process
 - relies on, enhances, network layer services
- **Socket address**
 - Combination of IP address and port number



Behrouz A. Forouzan, "Data Communications and Networking"

Transport vs. Network Layer – Example



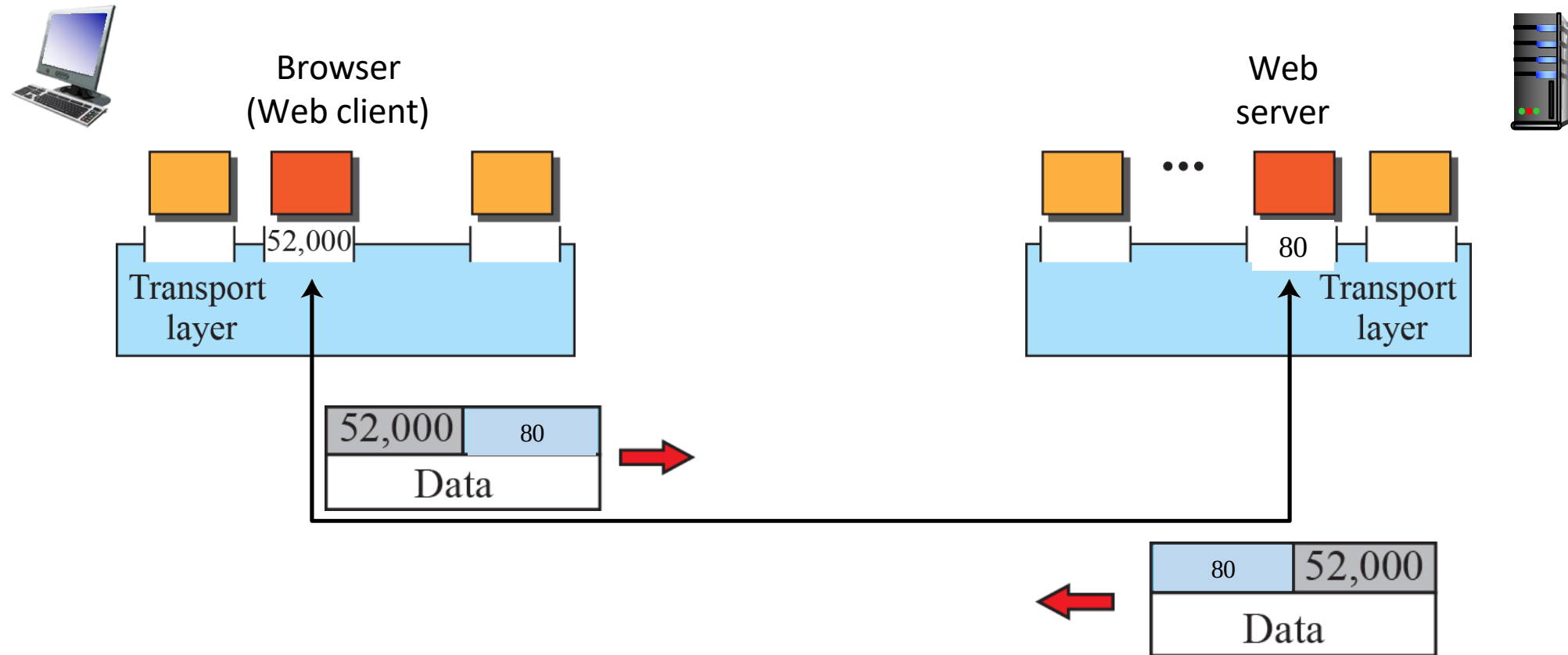
- Socket address: 193.14.26.7:13

Behrouz A. Forouzan, "Data Communications and Networking"

Port Number

- **16-bit** number
- **Internet Assigned Numbers Authority (IANA)** is responsible for maintaining the official assignments of port numbers for specific uses
 - <https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml>
- **Well-known** ports: 0 ~ 1023
 - used by system processes that provide *widely used types of network services*
- **Registered** ports: 1023 ~ 49151
 - assigned by IANA for specific service upon application by a requesting entity
- **Dynamic or private** ports: 49152 ~ 65535
 - used for private, or customized services or temporary purposes and for automatic allocation of **ephemeral** ports

Port Number – Example



Behrouz A. Forouzan, "Data Communications and Networking"

Problems Handled by Transport Protocols

- Connection management
 - Ensure that each party has agreed to communicate
- Error recovery (reliable data transfer)
 - Verify that data was not corrupted during transit
 - Detect and recover from datagram loss
 - Eliminate duplicate packets
 - Guarantee that messages arrive in order
 - Prevent delayed packets from being misinterpreted
- Flow control
 - Accommodate speed mismatch between sender and receiver
- Congestion control
 - Respond to congestion in the Internet
- Note: a given transport protocol may not handle all problems

Transport-Layer Protocols

- Two main protocols:
 - **TCP: Transmission Control Protocol**
 - **UDP: User Datagram Protocol**
- TCP: reliable, in-order delivery
 - **connection management**
 - **reliable data transfer (error recovery mechanism)**
 - **flow control**
 - **congestion control**
- UDP: unreliable, unordered delivery
 - no-frills extension of “best-effort” IP

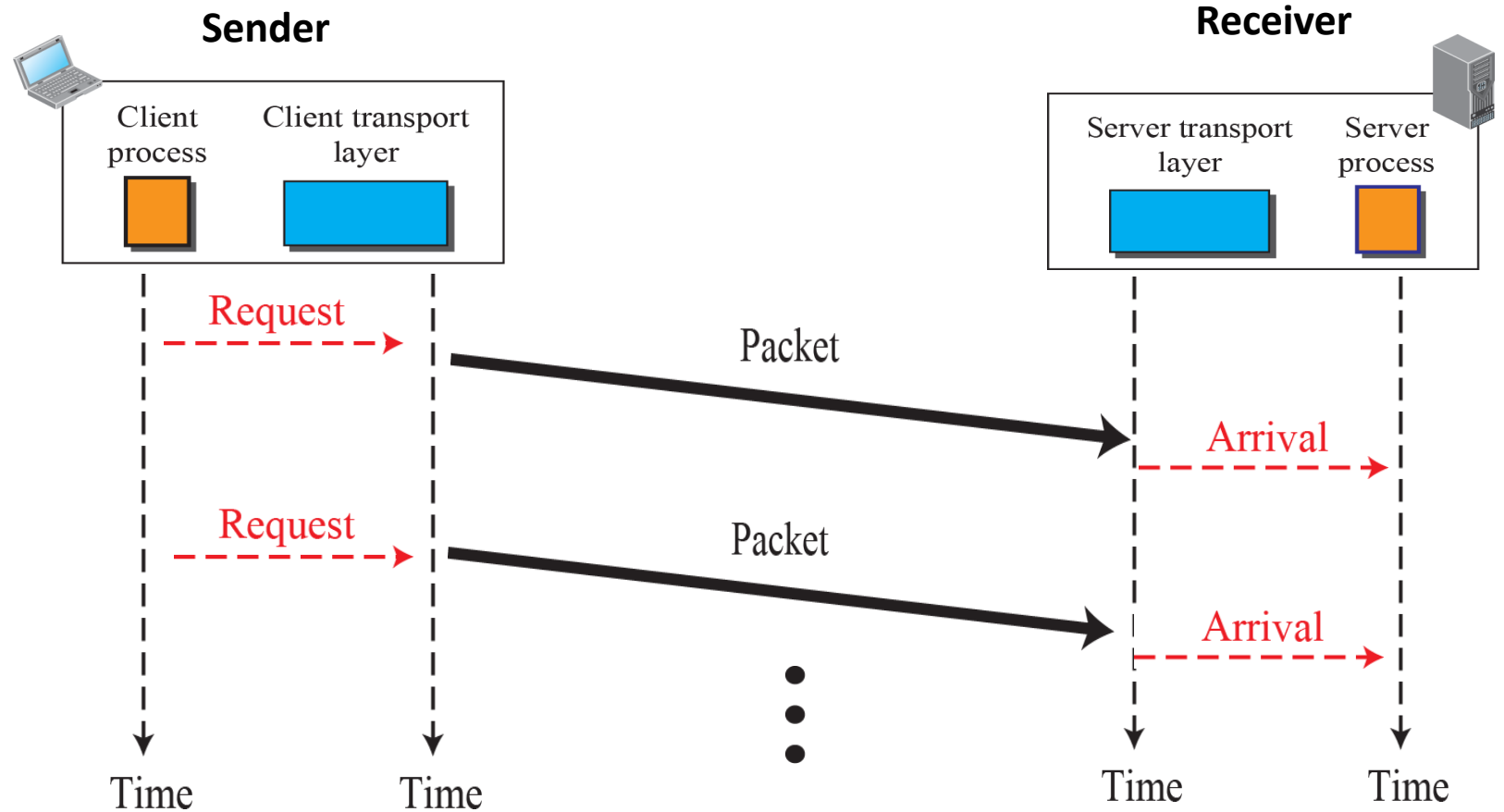
Contents

- Transport-layer services
- **Principles of reliable data transfer**
- Stop-and-Wait
- Go-Back-N
- Selective-Repeat



Transfer over Reliable Channel

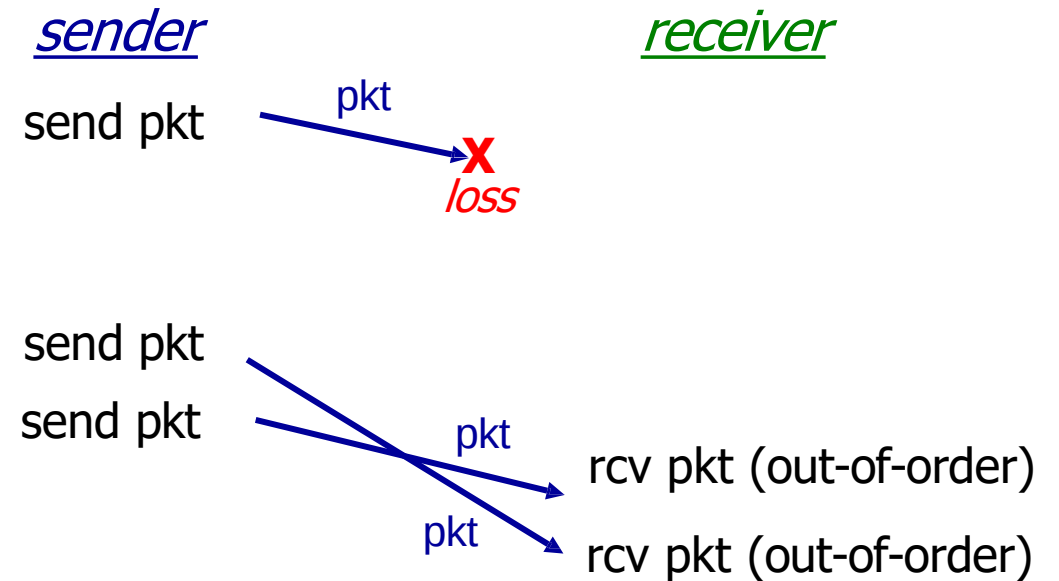
- **Ideal:** underlying channel *perfectly reliable*
 - no bit errors
 - no loss of packets
 - no out-of-order packets



Behrouz A. Forouzan, "Data Communications and Networking"

Transfer over Unreliable Channel

- Underlying channel may flip bits in packet
- May receive out-of-order packet
- Packets may loss during transmission



Reliable Transfer over Unreliable Channel

- Underlying channel may flip bits in packet
- May receive out-of-order packet
- Packets may loss during transmission
- Question: **How to *recover* from these errors in a network setting?**

How do humans recover from “errors” during conversation?

Reliable Transfer over Unreliable Channel (Cont.)

How do humans recover from “errors” during conversation?



<https://cdn.meme.am/instances/500x/68388526/despicable-me-minion-repeat-please.jpg>

Reliable Transfer over Unreliable Channel (Cont.)

How do humans recover from “errors” during conversation?



<https://cdn.meme.am/instances/500x/68388526/despicable-me-minion-repeat-please.jpg>

OK

Please
repeat that

Positive acknowledgments

- Sentence has been heard and understood

Negative acknowledgments

-Hears a garbled sentence. Ask to repeat

Reliable Transfer over Unreliable Channel (Cont.)

- Underlying channel may flip bits in packet
- May receive out-of-order packet
- Packets may loss during transmission
- Question: **How to *recover* from these errors in a network setting?**
- **Automatic Repeat reQuest (ARQ) protocol:**
 - *Error detection*
 - *Receiver feedback*
 - *Retransmission*

Contents

- Transport-layer services
- Principles of reliable data transfer
- **Stop-and-Wait**
- Go-Back-N
- Selective-Repeat

Stop-and-Wait – Key Techniques

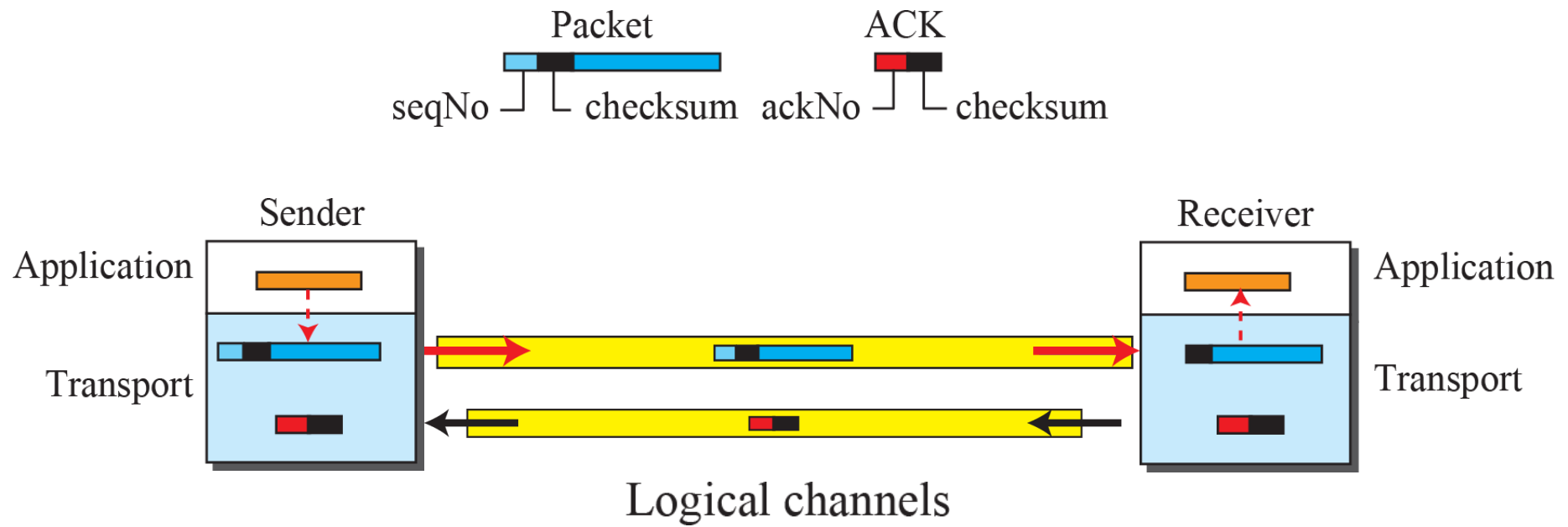
- **Checksum**
 - Detects bit errors
- **(Positive) Acknowledgment**
 - Verify that packet has been received
- **Timeout period (Timer)**
 - Protect against lost packet
 - Protect against lost acknowledgment
- **Sequence number**
 - Detects out-of-order packet
 - Detects duplicate packet
 - Detects duplicate acknowledgment



Stop-and-Wait

- Sender places **checksum** and **sequence number** in each packet
- Receiver uses *checksum* value to detect corrupted packet
- Receiver uses the *sequence numbers* to ensure that
 - no packets are missing
 - packets are delivered in the correct order
- Receiver sends **acknowledgement** to inform sender when a packet has been received correctly
- Sender **retransmits** packet if *acknowledgement fails* to arrive within a specified time (**timeout**)

Stop-and-Wait (Cont.)



Stop-and-Wait Operations

- Four possible scenarios
 - No loss
 - Packet corrupted / loss
 - Acknowledgment loss
 - Premature timeout / delayed acknowledgment

Stop-and-Wait – No Loss

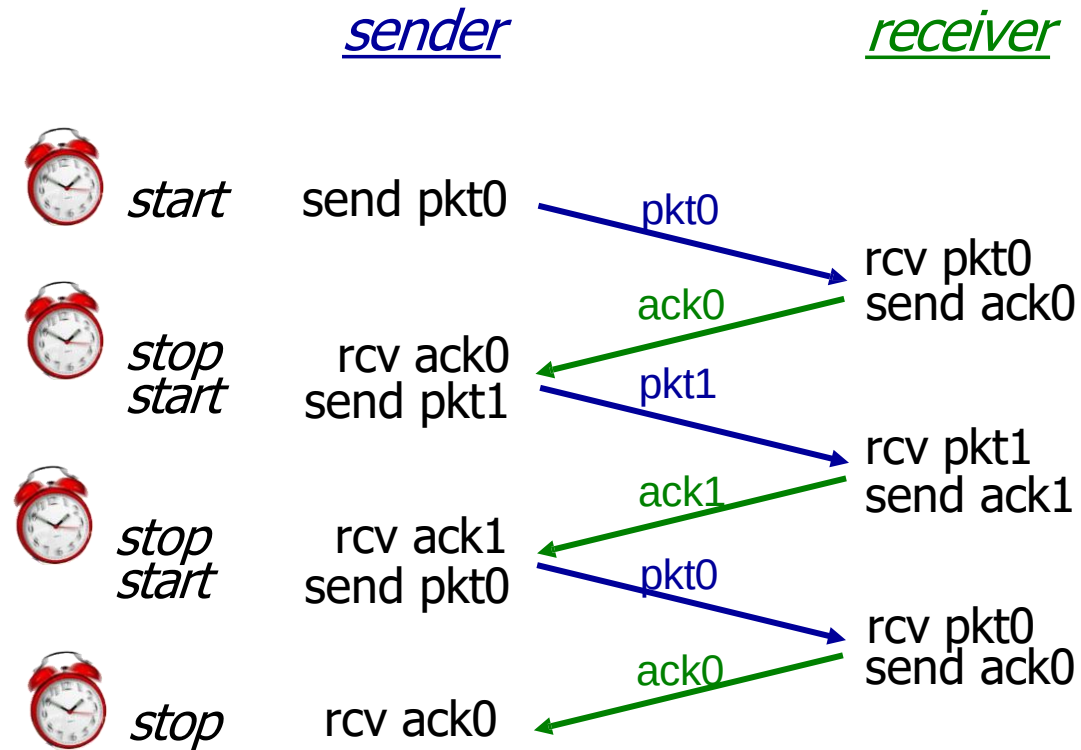
Sender

- Sends one packet at a time and waits for an *acknowledgment* before sending the next one.
 - places *checksum* in each packet
 - places *sequence number (0 or 1)* in each packet
- Starts timer
- Receives correct *acknowledgment* before timeout
- Stops timer

Receiver

- Checks the *checksum* upon receiving packet.
- If it is correct, sends *acknowledgement* to inform sender that packet received correctly.
 - ACK #: Specify *seq. #* of the received packet.

Stop-and-Wait – No Loss



Stop-and-Wait – Packet Corrupted / Loss

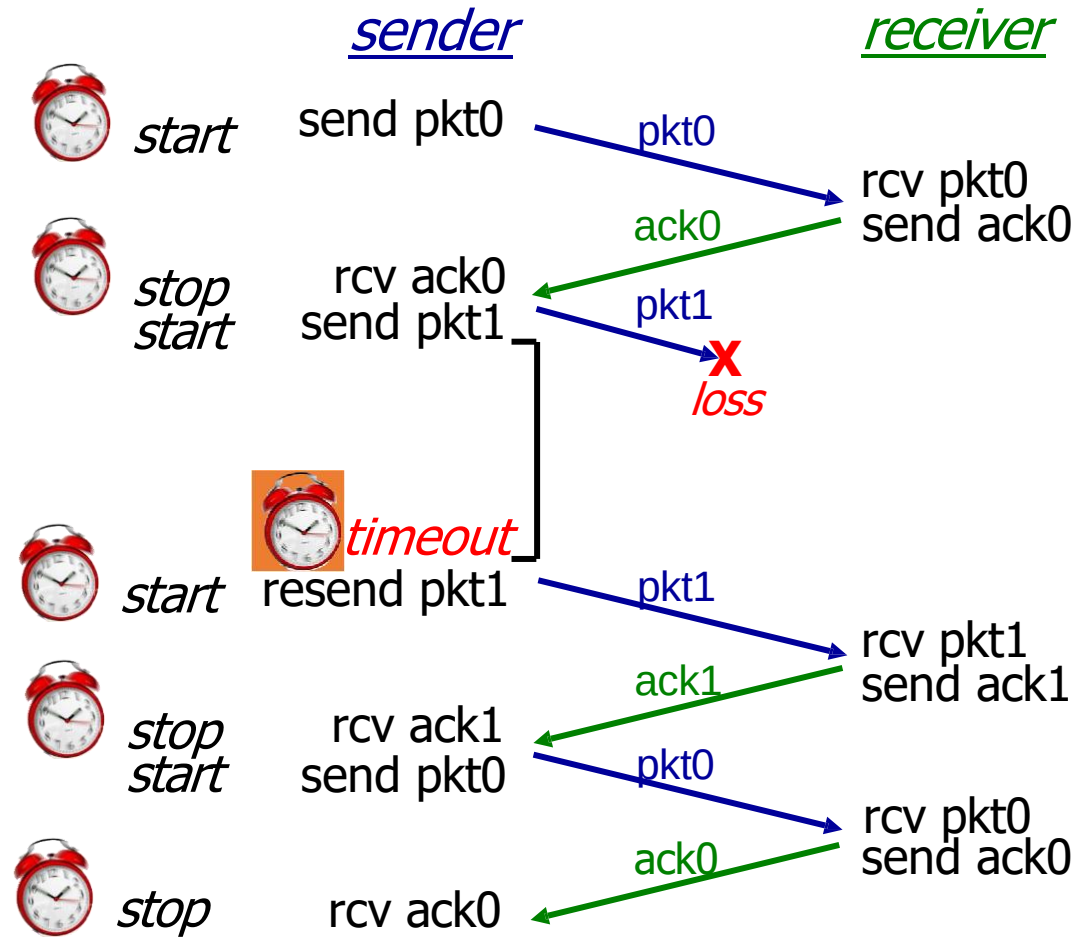
Sender

- Sends packet and waits for *acknowledgment*
- Starts timer
- **Resends** packet after *timeout*

Receiver

- Case 1: Checks the *checksum* upon receiving packet. If it is *incorrect*, the packet is corrupted and silently *discarded*.
- Case 2: Does not receive packet. (Packet *loss* during transmission)

Stop-and-Wait – Packet Corrupted / Loss



Stop-and-Wait – ACK Loss

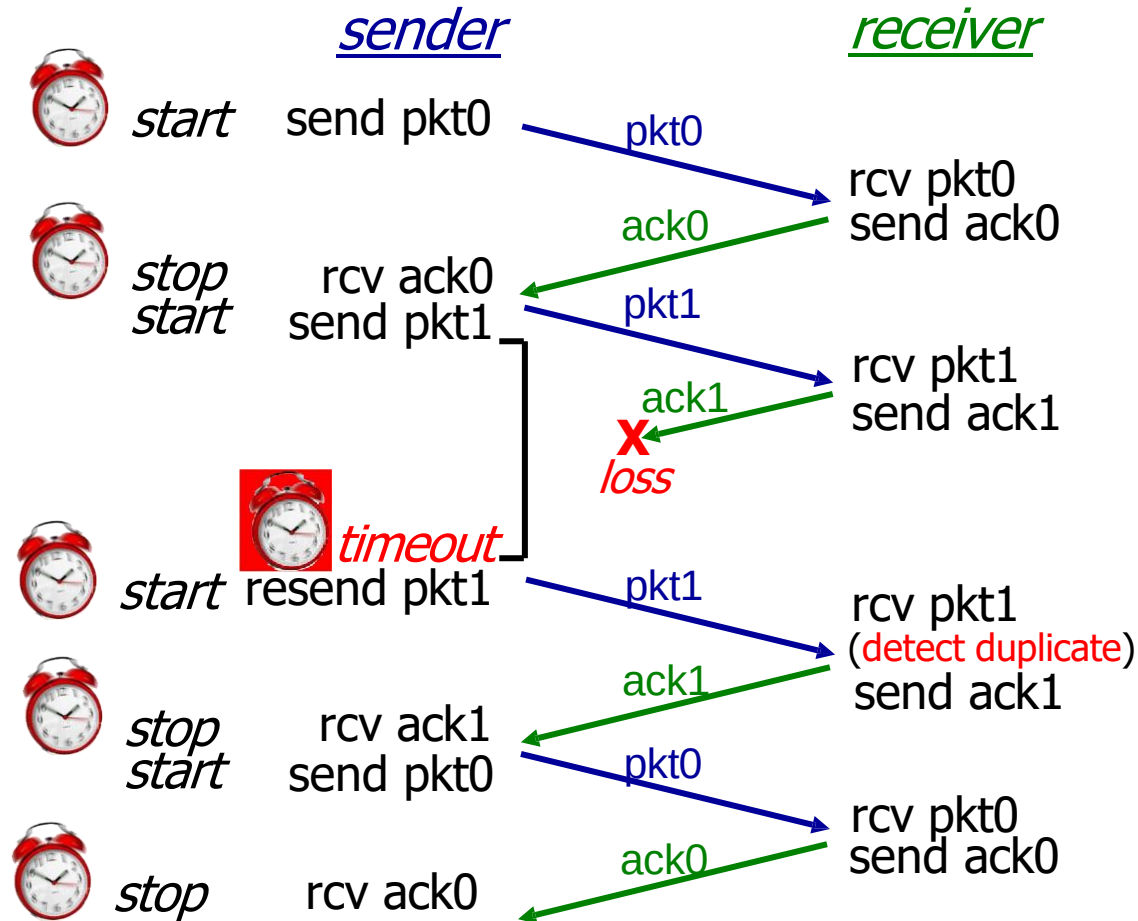
Sender

- Sends packet and waits for *acknowledgment*
- Starts timer
- Does not receive *acknowledgment* (*Acknowledgement loss*)
- *Resends* packet after *timeout*

Receiver

- Checks the *checksum* upon receiving packet. If it is correct, sends *acknowledgement*.
- Receives *duplicate* packet. *Discards* packet.
- Sends *acknowledgement*

Stop-and-Wait – ACK Loss



Stop-and-Wait – Premature Timeout / Delayed ACK

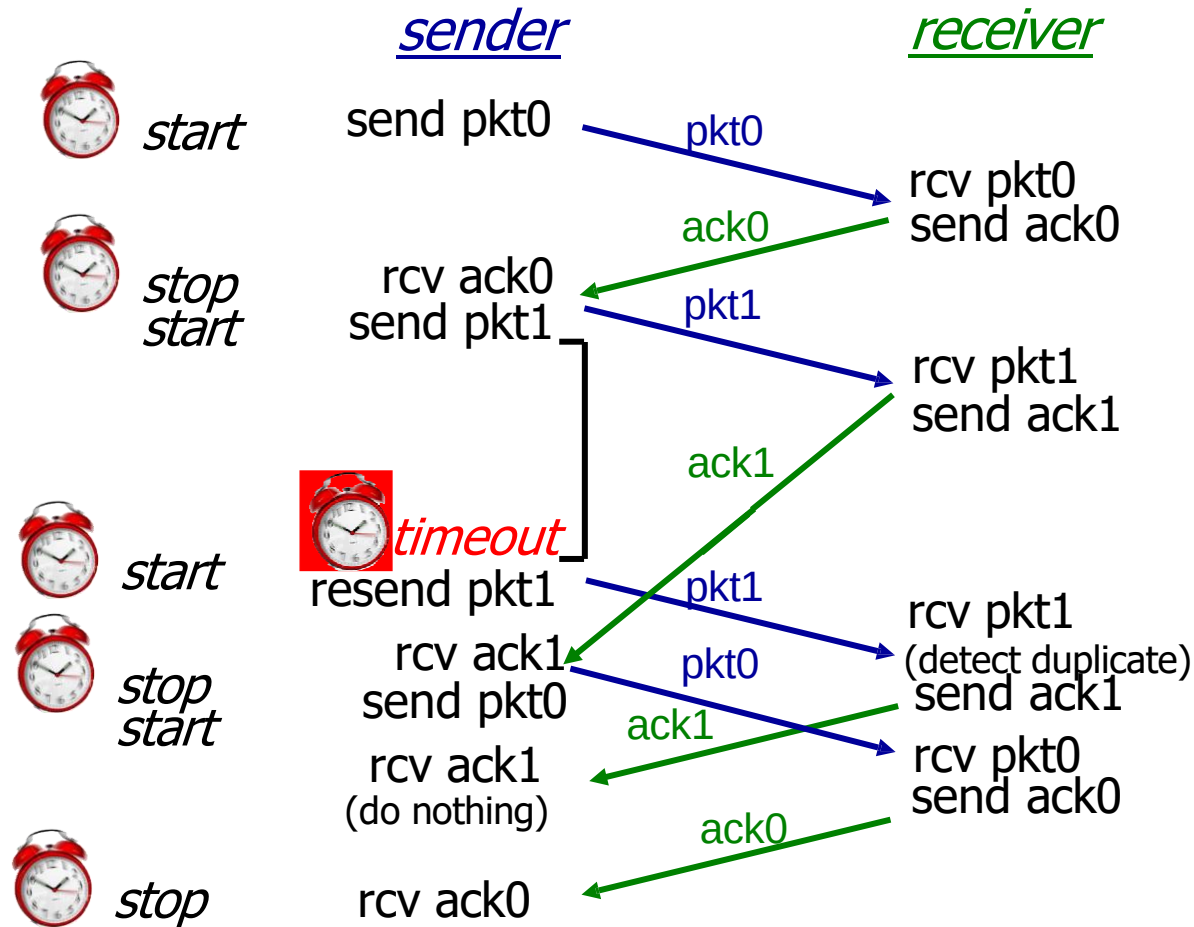
Sender

- Sends *pkt1* and waits for *acknowledgment* (Starts timer)
- Does not receive *ack1* (*ACK delays*)
- *Resends* *pkt1* after *timeout* (Starts timer)
- Receives delayed *ack1*. (Stops timer)
- Sends *pkt0* (Starts timer)
- Receives duplicate *ack1* (do nothing)

Receiver

- Receives *pkt1*, sends *ack1*
- Receives *duplicate* *pkt1*. *Discards* *pkt1*. Sends *ack1*

Stop-and-Wait ARQ – Premature Timeout / Delayed ACK



Limitation of Stop-and-Wait

- The protocol **limits use of available bandwidth**
- e.g.: 1 Gbps link, 15 ms prop. delay, 8000 bit packet:

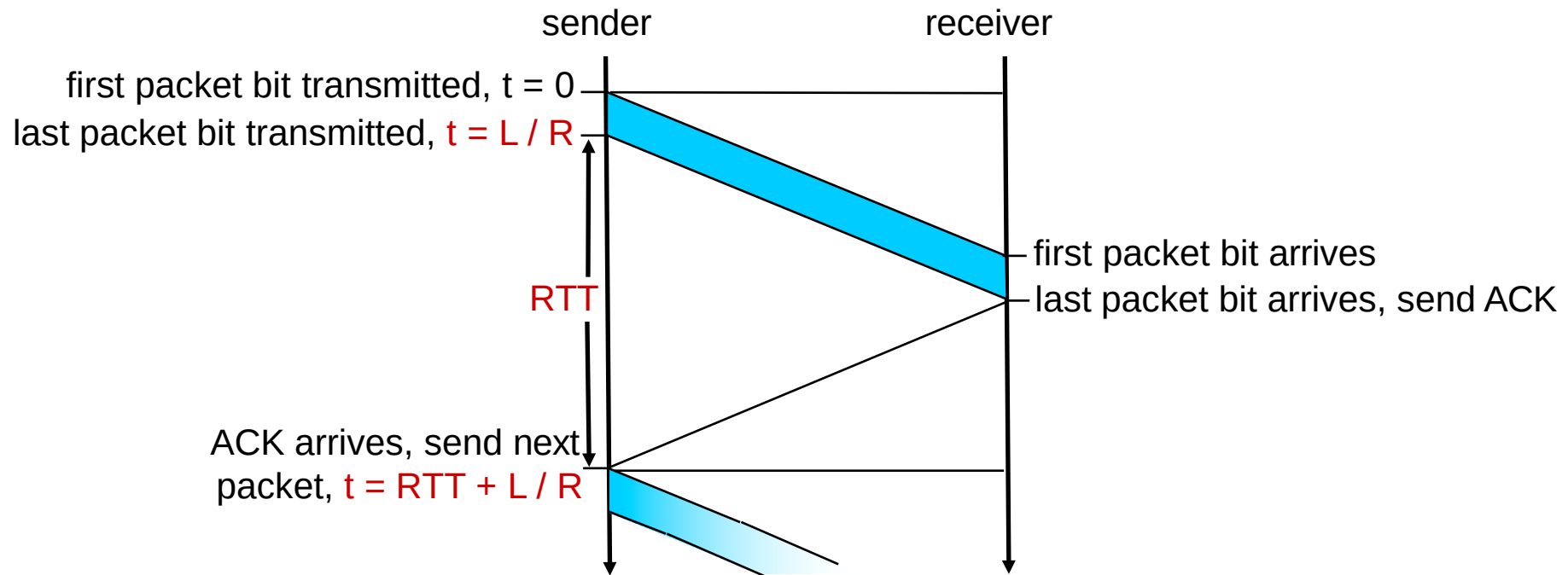
$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

- U_{sender} : **utilization** – fraction of time sender busy sending

$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$



Performance of Stop-and-Wait



$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

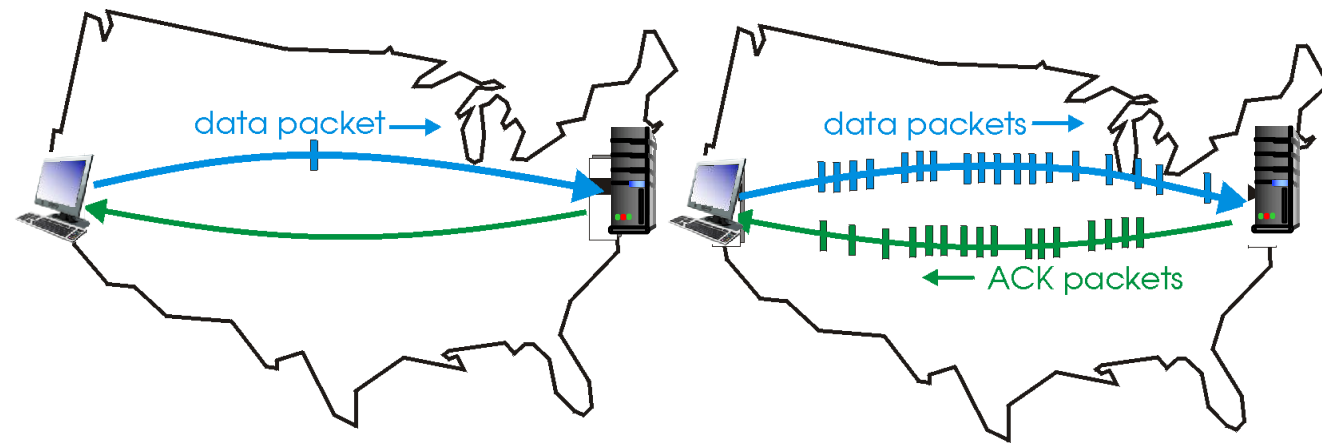
In 30.008 msec, sender was sending for only 0.008 msec

Contents

- Transport-layer services
- Principles of reliable data transfer
- Stop-and-Wait
- Go-Back-N
- Selective-Repeat

Pipelined Protocols

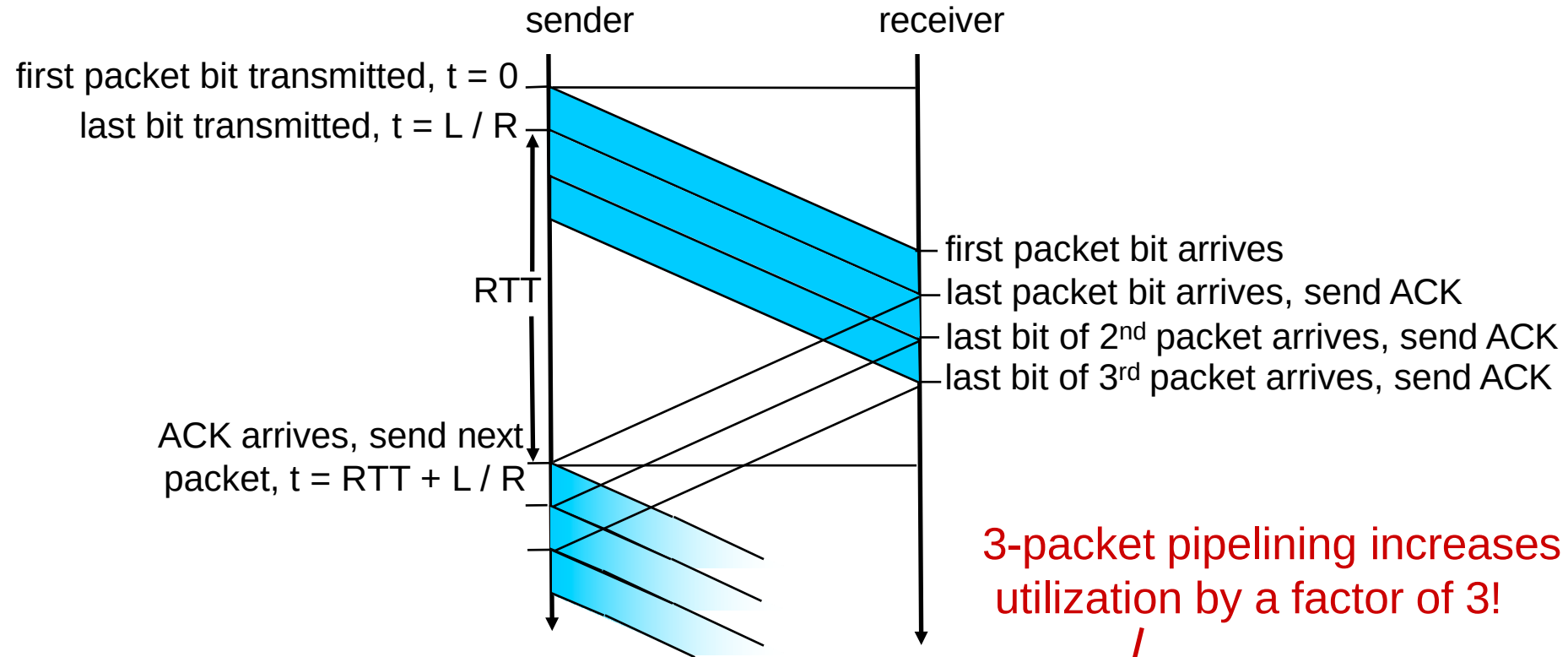
- **pipelining**: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts
 - *range* of sequence numbers must be *increased*
 - *buffering* at sender and/or receiver
- two generic forms of pipelined protocols:
 - **go-Back-N**
 - **selective repeat**



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

Pipelining: Increased Utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Pipelined Protocols: Overview

- Go-back-N:

- sender can have up to N unacked packets in pipeline
- receiver only sends *cumulative ack*
 - doesn't ack packet if there's a gap
- sender has timer for *oldest unacked* packet
 - when timer expires, retransmit *all unacked* packets

- Selective Repeat:

- sender can have up to N unacked packets in pipeline
- rcvr sends *individual ack* for each packet
- sender maintains timer for *each unacked* packet
 - when timer expires, retransmit *only that unacked* packet

Pipelined Protocols – Key Techniques

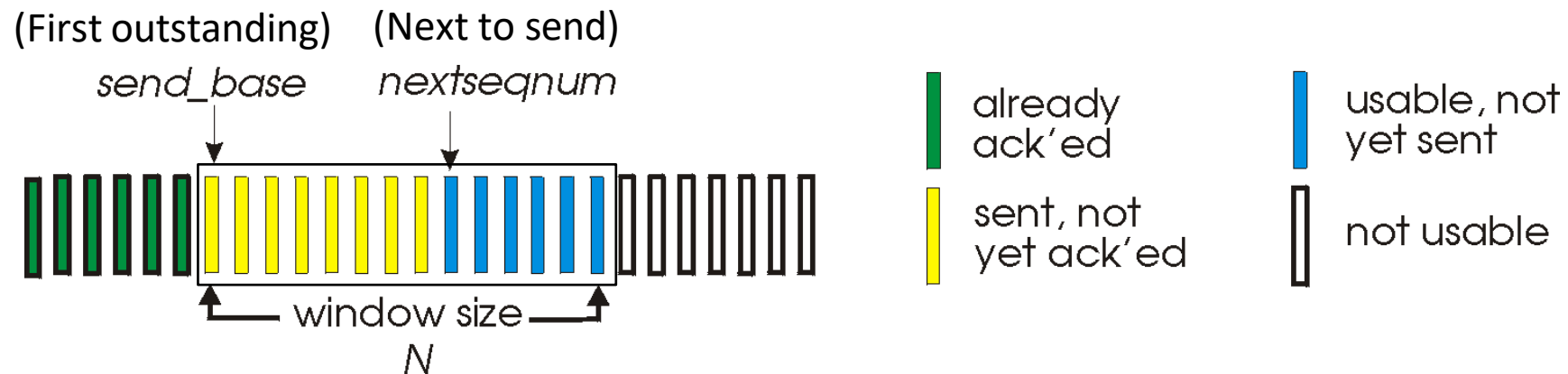
- Checksum
- (Positive) Acknowledgment
- Timeout period (Timer)
- Sequence number
- **Sliding window**
 - Allows multiple outstanding packets



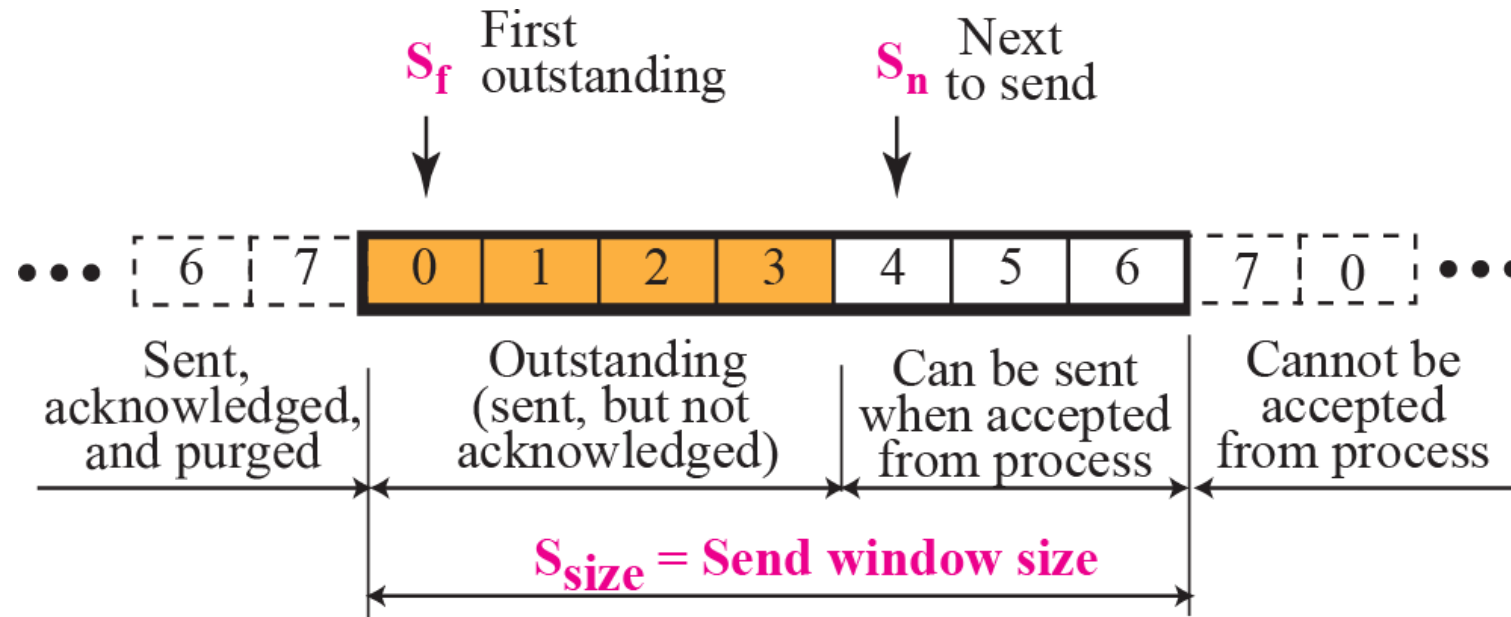
<http://www.china-windows.com/wp-content/uploads/2016/01/sliding-aluminum-alloy-window.jpg>

Go-Back-N: Sender's Window

- k -bit *seq #* in pkt header
- “window” of up to N , *consecutive unacked pkts* allowed



Example



Behrouz A. Forouzan, "Data Communications and Networking"

Go-Back-N: Sender

- $ACK(n)$: ACKs all pkts up to, including $seq \# n$ - “*cumulative ACK*”
 - may receive duplicate ACKs (see receiver)
- timer for *oldest* in-flight pkt
- $timeout(n)$: retransmit packet n and *all higher seq # pkts* in window

Go-Back-N: Receiver

- *ACK-only*: always send *ACK* for correctly-received *pkt* with *highest in-order seq #*
 - may generate duplicate ACKs
 - need only remember *rcv_base* (i.e., window size = 1)
- *out-of-order pkt*:
 - discard (don't buffer): *no receiver buffering!*
 - re-ACK *pkt* with highest in-order seq #



GBN In Action

sender window (N=4)

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

timeout(*n*):
 retransmit
 packet *n* and
all higher
 seq # pkts in
 window

sender

send pkt0
 send pkt1
 send pkt2
 send pkt3
 (wait)

rcv ack0, send pkt4
 rcv ack1, send pkt5

ignore duplicate ACK



pkt 2 timeout

send pkt2
 send pkt3
 send pkt4
 send pkt5

receiver

receive pkt0, send ack0
receive pkt1, send ack1

receive pkt3, discard,
 (re)send ack1

receive pkt4, discard,
 (re)send ack1

receive pkt5, discard,
 (re)send ack1

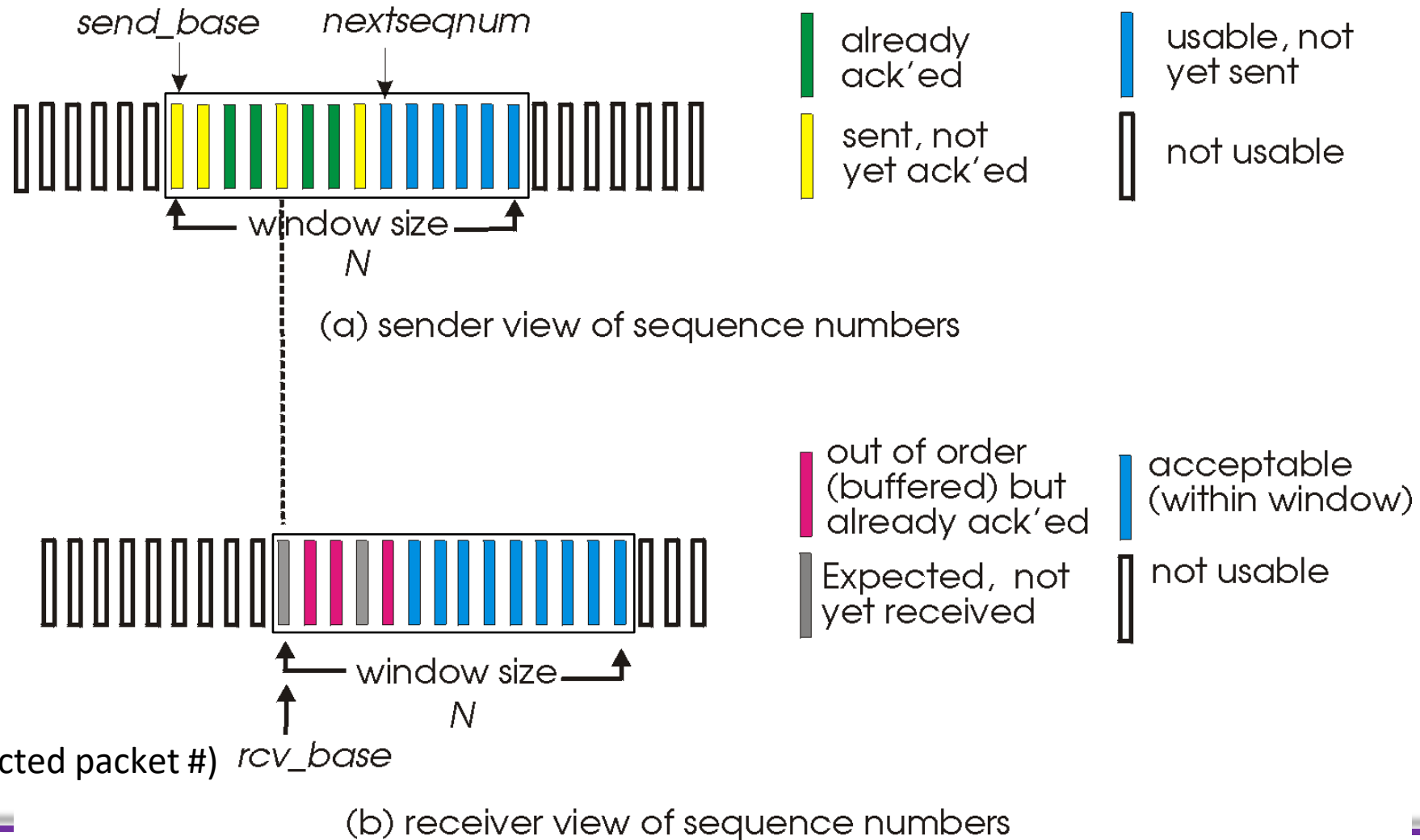
rcv pkt2, deliver, send ack2
 rcv pkt3, deliver, send ack3
 rcv pkt4, deliver, send ack4
 rcv pkt5, deliver, send ack5

always send ACK
 for correctly-
 received pkt with
 highest **in-order**
 seq #

Selective Repeat

- receiver *individually* acknowledges all correctly received pkts
 - *buffers pkts*, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
 - sender maintains *timer for each unACKed pkt*
- sender window
 - *N* consecutive seq #'s
 - limits seq #'s of sent, unACKed pkts

Selective Repeat: Sender, Receiver Windows



Selective Repeat

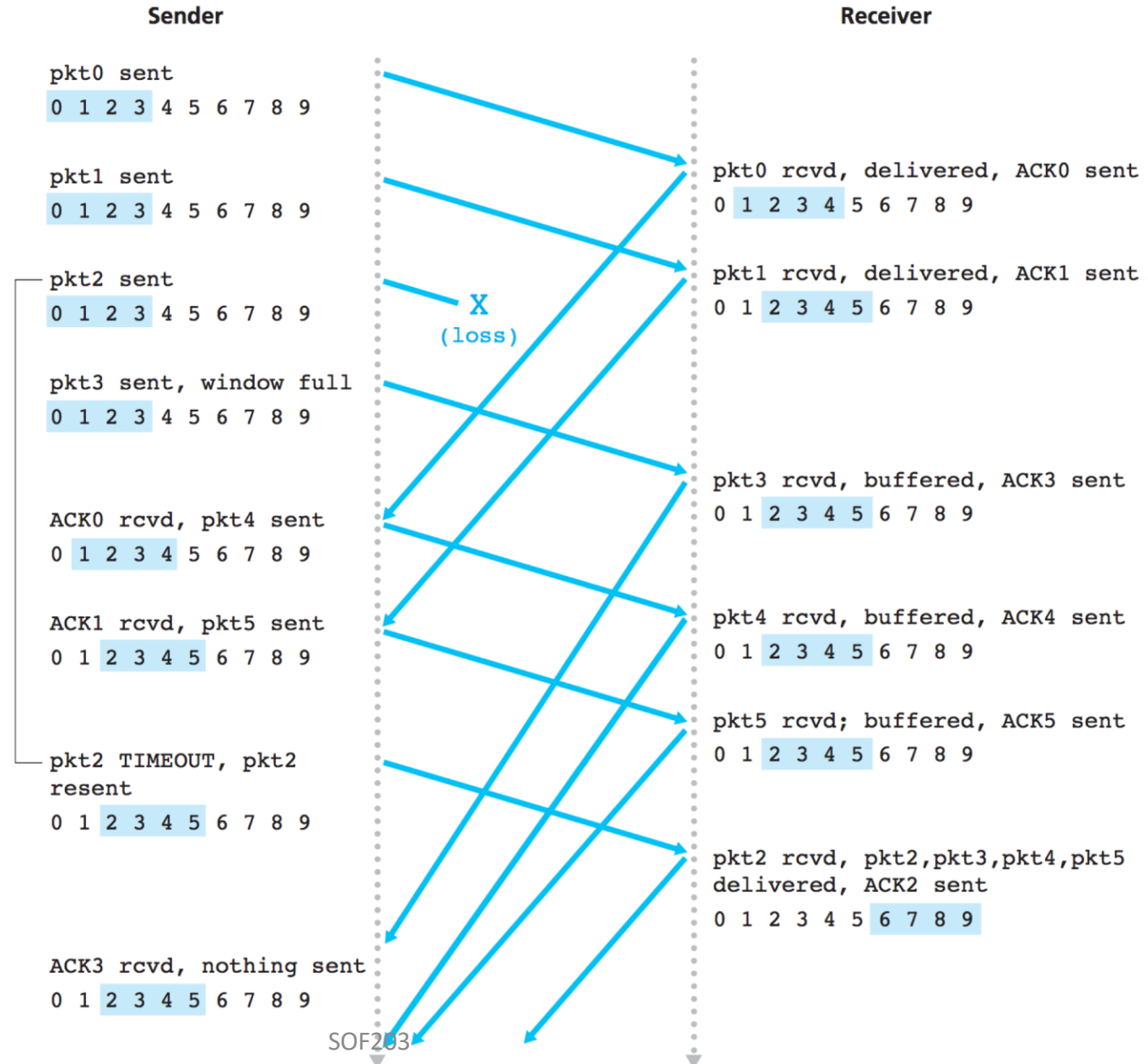
Sender

- data from above:
 - if next available seq # in window, send pkt
- **timeout(n):**
 - resend pkt n , restart timer
- **ACK(n) in [sendbase, sendbase+N-1]:**
 - mark **pkt n** as received
 - if **n smallest unACKed pkt**, advance window base to next unACKed seq #

Receiver

- **pkt n in [rcvbase, rcvbase+N-1]**
 - send **ACK(n)**
 - out-of-order: buffer
 - in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt
- **pkt n in [rcvbase-N, rcvbase-1]**
 - **ACK(n)**
- otherwise:
 - ignore

SR In Action



Interactive Animation

- Go-Back-N

https://media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/go-back-n-protocol/index.html

- Selective Repeat

https://media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/selective-repeat-protocol/index.html

Summary

- The primary way we achieve reliability over an unreliable channel is *automatic repeat request*
 - Explicitly notify sender of received packets by sending an *acknowledgement (ACK)*
 - *Resend* packets that are not ACKed within some timeframe
- We can make this efficient by having *many unACKed* packets simultaneously in flight
- *Sliding Window* is the standard technique to manage this process and keep the sender and receiver in sync
 - And limit the amount of data needing to be buffered

References

- Douglas E. Comer, “Computer Networks and Internets”
 - Chapter 24, 25
- James Kurose, Keith Ross, “Computer Networking: A Top-Down Approach”
 - Chapter 3



XIAMEN UNIVERSITY MALAYSIA

廈門大學 馬來西亞分校

Vision and Mission

Vision

Xiamen University Malaysia aspires to become a university with a distinct global outlook, featuring first-class teaching and research, and embracing cultural diversity.

Mission

To nurture young talents with dignity and wisdom, turning them into fine citizens of the region who will contribute to the prosperity of the people and social progress of Malaysia, China and Southeast Asia.

